

Appendix

A. Optimized Implementation of On-Device FFT Specifics

This appendix goes into depth on how to make the Fast Fourier Transform (FFT) work better for processing on devices. It also discusses the most important things that need to be in place for consistency and a complete technical review.

A.1. Selection Criteria and Reasons for Libraries

To perform Fast Fourier Transform (FFT) computations, we selected the KissFFT library (version 1.3.1) (Borgerding 2018). Several factors led to the use of this library. When resources are limited, this library is easy to use and works well. It has a simple C implementation with one header that is easy to add to and only needs 50 KB of memory. The ARM Cortex-A72 architecture also uses NEON SIMD instructions to speed things up. The library also uses NEON SIMD instructions to speed up operations on the ARM Cortex-A72 architecture. It is also capable of handling FFTs for real and complex data types of any size.

A.2. Implementation-Specific Details

A.2.1. Real-Time Constraints The signal processing framework uses a Hann window that is 25 ms long, or 400 samples at a rate of 16 kHz. We look at these frames with a hop size of 10 ms (160 samples), which means that each window overlaps by 60%. To make the frequency analysis easier, each 400-sample frame is padded with zeros to 512 samples before a 512-point Fast Fourier Transform (FFT) is done.

A.2.2. Memory Management A strategic memory management plan was put in place to make sure that data is handled quickly and performance stays the same. All of the input, output, and FFT twiddle factor buffers that are needed are set aside during initialization. This means that you don't need to allocate memory dynamically at runtime. By aligning these memory blocks to a 16-byte boundary, the aligned `aligned_alloc()` function makes them work with NEON SIMD instructions. This memory is great for streaming because it is set up as a circular buffer that can handle data that comes in all the time. This way of doing things completely stops dynamic memory allocation in the core processing loop. This stops memory fragmentation and makes latencies hard to predict.

A.3. Precision Analysis and Trade-offs

The implementation uses single-precision floating-point (FP32) math instead of double-precision (FP64) math. The decision was backed up by an accuracy test that showed that FP32 has enough accuracy for the job it was meant for. The highest relative error for a 512-point FFT tested on 10,000 random signals was less than $< 10^{-5}$. This level of accuracy is enough for 16-bit audio signals because it covers their 96 dB dynamic range and gives them an extra 6 bits of space. Since metrics like spectral leakage didn't get much better and the Signal-to-Noise Ratio (SNR) only went down by 0.1

dB compared to an FP64 reference version, FP32 was the better and more logical choice.

A.4. Detailed Performance Benchmarking

A.4.1. Method for Benchmarking The Raspberry Pi 4 Model B with a BCM2711 ARM Cortex-A72 processor running at a constant 1.5 GHz is the experimental platform. Following the compilation of the source code with GCC version 9.4.0, the optimization flags `--O3`, `-march=armv8-a+neon`, and `-mtune=cortex-a72` were added. To guarantee reliable and consistent results, each test was run 1000 times without thermal throttling. The cache hierarchy of the processor consists of a 256 KB L2 cache, a 32 KB L1 instruction cache, and a 32 KB L1 data cache. There is no L3 cache in this system.

A.4.2. Results of the Comparison A comparison of how different Fast Fourier Transform (FFT) implementations work shows how useful a specialized strategy can be. The KissFFT library is 1.48 times faster than NumPy's default double-precision implementation and uses less memory (from 8.2 KB to 5.6 KB). The main reasons for these improvements in efficiency are better algorithms and the smart use of single-precision floating-point (FP32) math. The findings indicate a compromise between precision and velocity in computations. The results show that there needs to be a balance between speed and accuracy in calculations. This shows that optimized single-precision FFTs work best on embedded systems with limited resources, where a small drop in accuracy can lead to big gains in performance.

Table 1: 512-point complex FFT performance comparison

Implementation	Execution Time (μ s)	Memory (KB)	Speedup
NumPy (FP64)	847.3 $\pm$ 12.1	8.2	1.0x
Standard KissFFT (FP32)	652.1 $\pm$ 8.9	6.1	1.30x
Optimized KissFFT (FP32)	572.8 $\pm$ 7.3	5.6	1.48x

B. Formulation of the Adaptive Feature Extraction Policy $\pi_{\theta}(x_t)$

B.1. Policy Definition and Motivation

The adaptive feature extraction policy $\pi_{\theta}(x_t)$ is a key part of StreamSplit. It changes the computational complexity of audio processing based on the resources of the edge device. This policy deals with the main problem of keeping audio processing going while dealing with different resource limits and lowering quality loss. The policy's official definition is:

$$\pi_{\theta}(x_t) = \begin{cases} \text{Full-resolution processing,} & \text{if } R_t \geq R_{threshold} \\ \text{Reduced-resolution processing,} & \text{otherwise} \end{cases} \quad (1)$$

where $R_{threshold}$ is the adaptive threshold that determines when to switch between processing modes, and R_t is the amount of resources available at time t (primarily CPU usage).

B.2. Resource Monitoring and State Representation

B.2.1. Multi-dimensional Resource Vector A multi-dimensional vector called R_t , which includes significant performance and status metrics, depicts the resource state of the system at any given time t . This state vector's official definition is:

$$R_t = [r_{cpu}(t), r_{mem}(t), r_{battery}(t), r_{thermal}(t)]^T. \quad (2)$$

This vector is made up of the CPU usage, $r_{cpu}(t)$, and the memory usage, $r_{mem}(t)$. Both are shown as numbers between 0% and 100%. It also shows the battery level left, $r_{battery}(t)$, as a percentage. If the value is 1, the device is getting too hot; if the value is 0, it is in a normal thermal state. The last part, $r_{thermal}(t)$, is a binary indicator of the system's temperature.

B.2.2. Resource Aggregation Function To make the main implementation easier, the multi-dimensional resource state vector is turned into a single scalar value, R_t , with a weighted aggregation function. The formula below shows how this function combines the different system metrics to make a single score:

$$R_t = w_{cpu} \cdot r_{cpu}(t) + w_{mem} \cdot r_{mem}(t) + w_{battery} \cdot r_{battery}(t) + w_{thermal} \cdot r_{thermal}(t). \quad (3)$$

The default weights for each part are: $w_{cpu} = 0.5$, $w_{mem} = 0.3$, $w_{battery} = 0.15$, and $w_{thermal} = 0.05$. Through extensive system profiling, these specific values were empirically determined to best represent the relative importance of each resource.

B.3. Adaptive Threshold Mechanism

The resource threshold, or $R_{threshold}$, can be set in two different ways. A static threshold equal to 70% of the CPU usage is used in the primary implementation. This value was obtained by methodically profiling various audio processing workloads on the Raspberry Pi 4B platform in order to identify a strong operational limit.

To make it simpler to adapt to novel circumstances, an alternative approach that continuously modifies the threshold is recommended. An exponential smoothing filter modifies the threshold at each time step t to achieve this:

$$R_{threshold}^{(t+1)} = \alpha \cdot R_{threshold}^{(t)} + (1 - \alpha) \cdot f(R_t, Q_t) \quad (4)$$

In this model, $f(R_t, Q_t)$ is an adaptation function that provides a target value based on the current state of the system, and α is a smoothing factor set to 0.9. Depending on whether the recent processing quality (Q_t) satisfies a target (Q_{tgt}), the function modifies the threshold:

$$f(R_t, Q_t) = R_{th}^{(t)} + \begin{cases} +\delta_{inc}, & Q_t > Q_{tgt} \wedge R_t < R_{th}^{(t)} \\ -\delta_{dec}, & Q_t < Q_{tgt} \wedge R_t > R_{th}^{(t)} \\ 0, & \text{otherwise} \end{cases} \quad (5)$$

The threshold increases by $\delta_{inc} = 2\%$ when performance is high and resources are not being used to their full potential. The system decreases by $\delta_{dec} = 1\%$ when performance is poor and it is overloaded.

B.4. Processing Mode Specifications

The system has a dual-mode processing strategy that changes based on a set threshold, $R_{threshold}$, and the current state of the resources, R_t . When the resource load is below the threshold ($R_t < R_{threshold}$), the system works in a default, full-resolution mode. This standard mode uses a 400-sample (25 ms) analysis window with a 10 ms hop size (160 samples) to make a full 257-bin spectrum. A complete mel-spectrogram with 128 frequency bins is produced as a result.

In contrast, the system transitions to a lower-resolution mode to conserve processing power when the resource load hits or exceeds the threshold ($R_t \geq R_{threshold}$). Several optimization techniques are used to achieve this:

- *Frequency Domain Reduction*: The frequency band of interest is limited to the range of [80Hz, 8kHz]. The number of mel bins is cut in half to 64, and sparse FFT algorithms are used to make the calculations work better.
- *Changing the temporal resolution*: By doubling the hop size to 320 samples (20 ms), the rate at which the time updates happens is slowed down. To avoid artifacts from the temporal downsampling, an anti-aliasing filter is used, and the original window size is kept to keep the frequency resolution.
- *Computational Optimizations*: Using lookup tables to speed up expensive logarithmic calculations, switching from floating-point to integer arithmetic for tasks like mel-scale conversion, and getting rid of the calculation of non-essential higher-order spectral features all help make things run more quickly.

B.5. Quality Preservation Mechanisms

Q_t is a composite perceptual quality metric that gives a number to audio fidelity. By using a weighted sum to combine three important metrics into one score, this metric gives a full picture of the quality of the output. The official definition of the metric is as follows:

$$Q_t = w_1 \cdot SNR_t + w_2 \cdot (1 - THD_t) + w_3 \cdot STFT_{coherence} \quad (6)$$

The Total Harmonic Distortion (THD_t) and the Signal-to-Noise Ratio (SNR_t) are both important. The second one is flipped so that the score is better with less distortion. The last term, $STFT_{coherence}$, looks at how similar the spectra of the full and reduced-resolution outputs are to see if any processing changed anything.

When the system is in its low-resolution, resource-saving mode, a set of adaptive compensation techniques is used to actively stop quality from getting worse. One of these is Dynamic Range Compression. There is a modified dynamic range that keeps important spectral features. If there are missing frequency bins, spectral interpolation fills them in. This is usually done with cubic splines for accuracy. Finally, temporal smoothing gets rid of temporary artifacts that might happen when adaptive processing changes. A median filter across frames is usually used for this.

B.6. Implementation Details

The resource monitoring collects samples every 100ms while operating on a separate thread:

```

1 // Resource monitoring
2 struct ResourceState {
3     float cpu_util, mem_usage,
4         battery_level;
5     bool thermal_throttling;
6     uint64_t timestamp;
7 };
8 ResourceState monitor_resources() {
9     ResourceState state;
10    state.cpu_util = get_cpu_utilization();
11    state.mem_usage = get_memory_usage();
12    state.battery_level = get_battery_level();
13    state.thermal_throttling = check_thermal_state();
14    state.timestamp = get_timestamp_us();
15    return state;
16 }
```

A hysteresis mechanism in the control logic prevents the full and reduced-resolution modes from abruptly and unevenly switching between them. Rather than using a single switching point, this approach establishes distinct upper and lower bounds for state transitions. Only when the resource metric R_t falls below a specific threshold, known as $R_{threshold} - h_{down}$, will the system transition to the low-resolution, resource-saving mode. However, before returning to full-resolution mode, the metric needs to recover and cross an upper threshold of $R_{threshold} + h_{up}$. The upward margin (h_{up}) and downward margin (h_{down}) are set at 3% and 5%, respectively, for stable operation. As a result, the system has a sizable buffer that prevents random on/off switching.

B.7. Experimental Validation

To determine the optimal value, we performed a sensitivity analysis by varying the static resource threshold ($R_{threshold}$) from 60% to 80%. The outcomes demonstrated a definite trade-off between obtaining high-quality audio and conserving resources. The low threshold of 60% was too aggressive, resulting in frequent mode switches and a 2.3% quality loss. Conversely, an 80% threshold provided nearly no adaptive benefit. However, a high threshold of 75% was overly cautious and only resulted in a 0.9% improvement in quality at the expense of 15% more resources. The best compromise was ultimately found to be a 70% threshold, which resulted in notable computational savings and a tolerable 1.2% quality drop.

Then, using one-hour continuous audio streams, we experimented with three different settings: the recommended adaptive mode that employed the 70% threshold, a static full-resolution mode, and a static reduced-resolution mode. The results showed how successful the adaptive strategy was. Because of the adaptive system, the battery lasted 50% longer (from 3.2 to 4.8 hours) and the CPU usage dropped

(from 86.5% to 65.2%). To achieve this significant increase in efficiency, the perceptual quality score only needed to decrease marginally, from 0.95 to 0.93. The quality was noticeably worse even though the “reduced-resolution only” mode consumed the fewest resources (0.87). By maintaining a high level of audio quality while reaping the majority of the power-saving advantages, the adaptive system successfully balanced the objectives. It switched modes twenty-three times during the test.

Table 2: Processing mode comparison

Metric	Full-Res Only	Adaptive (70%)	Reduced-Res Only
Avg CPU (%)	86.5	65.2	42.7
Quality Score	0.95	0.93	0.87
Battery Life (hrs)	3.2	4.8	6.1
Mode Switches	0	23	0

C. Implementation Details of Reduced-Resolution Processing

When available CPU utilization R_t falls below 70%, the system enters reduced-resolution processing mode. While maintaining audio quality sufficient for edge-based contrastive learning, this adaptive mechanism ensures that the system continues to function even in the event of resource constraints. In order to preserve crucial spectral and temporal features required for subsequent representation learning tasks, the reduced-resolution processing makes use of meticulously designed optimizations.

C.1. Frequency Domain Reduction

C.1.1. Mel-Scale Bin Reduction The main change in the frequency domain is to cut the number of mel-scale bins from 128 to 64. This reduction is not just done by down-sampling; it is done by intelligent bin averaging:

$$M'_i = \frac{1}{2} (M_{2i} + M_{2i+1}), \quad i = 0, 1, \dots, 63, \quad (7)$$

where M'_i is the i -th reduced mel bin and M_j is the original mel bin. This averaging makes the calculations easier while keeping the overall spectral envelope.

C.1.2. Perceptually-Guided Frequency Band Limiting

For environmental audio analysis, we only consider frequencies between 80 Hz and 8 kHz because that is where the most perceptually significant content is found.

$$f_{low} = 80 \text{ Hz}, \quad f_{high} = 8000 \text{ Hz} \quad (8)$$

The mel-scale limits are as follows:

$$m_{low} = 2595 \log_{10} \left(1 + \frac{80}{700} \right) \approx 127.14 \text{ mel} \quad (9)$$

$$m_{high} = 2595 \log_{10} \left(1 + \frac{8000}{700} \right) \approx 2840.02 \text{ mel} \quad (10)$$

C.1.3. Sparse FFT Implementation We use a sparse FFT that only calculates the frequency bins that are needed to save time and money:

Algorithm 1: Sparse FFT for Reduced-Resolution Processing

```

1: Input: Audio segment  $x[n]$ , frequency range  $[f_{low}, f_{high}]$ 
2: Output: Sparse frequency domain representation  $X_{sparse}[k]$ 
3: Compute relevant bin indices:  $k_{start} = \lfloor \frac{f_{low} \cdot N}{f_s} \rfloor$ ,  $k_{end} = \lceil \frac{f_{high} \cdot N}{f_s} \rceil$ 
4: Apply windowing:  $x_w[n] = x[n] \cdot w[n]$  where  $w[n]$  is Hann window
5: Initialize  $X_{sparse}[k] = 0$  for  $k \in [k_{start}, k_{end}]$ 
6: for  $k = k_{start}$  to  $k_{end}$  do
7:   for  $n = 0$  to  $N - 1$  do
8:      $X_{sparse}[k] += x_w[n] \cdot e^{-j2\pi kn/N}$ 
9:   end for
10: end for
11: return  $X_{sparse}$ 

```

This method cuts the computational complexity from $\mathcal{O}(N \log N)$ to about $\mathcal{O}(B \cdot N)$, where B is the number of relevant frequency bins ($B \ll N$).

C.2. Temporal Resolution Adjustment

Reducing the feature frame rate by dynamically altering the processing stride is the primary temporal optimization method. With a decimation factor of $d_s \cdot s = 2$, the analysis hop size (s) is specifically doubled from the typical 160 samples to 320 samples, as the following equation illustrates:

$$s' = d_s \cdot s = 2 \cdot 160 = 320 \text{ samples.} \quad (11)$$

With this modification, the analysis window size remains at 400 samples (25 ms), but the temporal resolution is cut in half. This design decision is crucial because it ensures that the spectral analysis's frequency resolution remains constant.

First, a well-crafted low-pass anti-aliasing filter is used to prevent temporal aliasing artifacts from this downsampling. The impulse response of the windowed sinc function filter is:

$$h_{AA}[n] = \frac{\sin(\pi n/d_s)}{\pi n/d_s} \cdot w_{Kaiser}[n]. \quad (12)$$

In this implementation, a Kaiser window ($w_{Kaiser}[n]$) with a shape parameter of $\beta = 6.5$ shapes the ideal low-pass filter. Because it provides the best time-frequency localization for audio signals, this shape was selected.

Finally, to ensure that significant temporal dynamics are maintained, the system explicitly computes and maintains the signal's spectral envelope. This is particularly crucial for later contrastive learning activities. We accomplish this by using a modified short-time energy function, which is computed as follows for every frame m :

$$E[m] = \sum_{n=0}^{L-1} |X[m, n]|^2 \cdot w_{env}[n]. \quad (13)$$

More recent samples within the analysis window are given more weight by the exponential weighting function denoted

by the term $w_{env}[n]$. This enables the function to record the crucial temporal envelope data despite the reduced frame rate.

C.3. Computational Optimizations

To optimize computational performance, a number of strategies are used to lower the overhead of floating-point calculations. Fixed-point arithmetic is used to convert frequency values to the mel scale in order to expedite processing on the target ARM architecture. The standard formula is multiplied by 2^{16} and then rounded to maintain the value's 16-bit fractional part while maintaining sufficient accuracy. The definition of the calculation is:

$$m_{fixed} = \text{round}(2595 \cdot \log_{10}(1 + f/700) \cdot 2^{16}) \quad (14)$$

Additionally, by storing frequently needed results, pre-computed lookup tables (LUTs) speed up computations. There are three primary LUTs in use:

- The numbers 1 through 100 are covered by a logarithm LUT with 4096 entries. For values in between, linear interpolation is employed.
- It is simple to switch back to frequency from the mel scale with a Mel-to-Hz LUT that has 256 entries.
- By storing the Hann window's pre-calculated coefficients in a Window Function LUT, expensive real-time trigonometric calculations during signal framing are avoided.

C.4. Quality Assessment and Validation

A set of spectral distortion metrics is used to measure the quality preservation of the reduced-resolution processing mode. These consist of objective metrics for magnitude and phase distortion, along with a standardized perceptual assessment instrument. The exact metrics are as follows:

$$SD_{mag} = \frac{1}{K} \sum_{k=0}^{K-1} (|X_{full}[k]| - |X_{reduced}[k]|)^2 \quad (15)$$

$$SD_{phase} = \frac{1}{K} \sum_{k=0}^{K-1} |\angle X_{full}[k] - \angle X_{reduced}[k]|^2 \quad (16)$$

$$PESQ = f_{PESQ}(x_{full}, x_{reduced}) \quad (17)$$

The mean squared error between the full and reduced-resolution signals in the magnitude and phase domains is displayed in this instance by SD_{mag} and SD_{phase} , respectively. The Perceptual Evaluation of Speech Quality (PESQ) score is a common way to tell how good something sounds.

An empirical analysis was performed on various audio samples utilizing these metrics to validate the efficacy of the reduced-resolution mode. The results show that it works great with a lot of different audio formats. The system did a great job with environmental sounds, getting a high PESQ score of 4.05, a spectral correlation of 0.97, and a very small Signal-to-Noise Ratio (SNR) loss of only 0.6 dB. The degradation was a little worse for more complicated signals, like music. The PESQ score was 3.85 and the SNR loss was 1.2 dB. The speech and mixed content performances covered this range, proving that the optimization strategies successfully preserve high perceptual quality across a range of audio categories.

Table 3: Quality metrics for reduced-resolution processing across different audio types

Audio Type	SNR Loss (dB)	PESQ Score	Spectral Correlation
Speech	0.8	3.92	0.96
Music	1.2	3.85	0.94
Environmental	0.6	4.05	0.97
Mixed Content	1.0	3.88	0.95

C.5. Memory Management Optimizations

Calculations are performed in-place whenever possible to reduce the system’s memory requirements. By substituting the output of an operation for the input data, this technique eliminates the need for separate output buffers and is excellent for element-wise tasks like frequency-domain filtering. This is how an in-place operation appears:

$$X[k] \leftarrow X[k] \cdot H[k], \quad k = 0, 1, \dots, N - 1 \quad (18)$$

Compared to functionally equivalent out-of-place implementations, the system’s peak memory usage decreases by roughly 40% when this technique is used consistently.

To process continuous, real-time audio data rapidly, the system makes use of circular buffers with improved indexing. This technique reduces the overhead associated with dynamic memory allocation by processing streams without gaps inside a fixed-size memory block. The fundamental concept of handling the write and read pointers for incoming samples and outgoing frames is explained by the algorithm that follows.

Algorithm 2: Circular Buffer for Streaming Audio

```

1: Initialize: Buffer  $B[N]$ , write pointer  $w\_ptr = 0$ , read pointer  $r\_ptr = 0$ 
2: for each audio sample  $x[n]$  do
3:    $B[w\_ptr] = x[n]$ 
4:    $w\_ptr = (w\_ptr + 1) \bmod N$ 
5:   if buffer is full then
6:     Process frame starting at  $r\_ptr$ 
7:      $r\_ptr = (r\_ptr + \text{hop\_size}) \bmod N$ 
8:   end if
9: end for

```

To ensure a constant flow of data for real-time analysis, modulo arithmetic is utilized to ensure that the pointers smoothly wrap around the buffer boundaries.

C.6. Adaptive Trade-off Parameters

An adaptive parameter known as $\alpha_{quality}$ is employed to actively regulate the balance between computational efficiency and perceptual quality. This parameter gives a normalized measure of the system resources that are available ($R_{available}$) in relation to a specified minimum (R_{min}) and an operational threshold ($R_{threshold}$). This is how to figure it out:

$$\alpha_{quality} = \frac{R_{available} - R_{min}}{R_{threshold} - R_{min}}, \quad \alpha_{quality} \in [0, 1]. \quad (19)$$

The final value of α quality directly affects how aggressive the data reduction optimizations are. These relationships are

used to change the decimation factors for frequency (d_f) and time (d_s). This lets the system use more aggressive down-sampling when resources are low and lower the optimization strength as resources become more available:

$$d_f = 2 - \alpha_{quality} \quad (\text{frequency decimation}) \quad (20)$$

$$d_s = 2 - 0.5 \cdot \alpha_{quality} \quad (\text{temporal decimation}) \quad (21)$$

In addition to this immediate, resource-based control, the system has a way to change its internal parameters over time. This change uses a gradient-based update rule to reduce a certain quality loss function ($L_{quality}$) and is based on recent processing statistics. To change the generic system parameter β adapt at each time step t , use the following equation:

$$\beta_{adapt}^{(t+1)} = \beta_{adapt}^{(t)} + \eta \cdot \frac{\partial L_{quality}}{\partial \beta_{adapt}^{(t)}} \quad (22)$$

In this case, the gradient term shows the direction of change that will most improve the quality score. The update’s size is controlled by a very small learning rate, $\eta = 0.01$, which ensures that convergence happens slowly and steadily over time.

C.7. Performance Analysis

According to a formal complexity analysis, the reduced-resolution processing mode saves a lot of processing power all along the pipeline. A sparse implementation of the standard Fast Fourier Transform makes the spectral analysis stage less complicated, going from $\mathcal{O}(N \log N)$ to $\mathcal{O}(B \cdot N)$. In this case, B stands for the number of relevant frequency bins. This cuts the amount of work needed for this job by 65%. The mel filtering stage’s workload is cut in half when the number of bins is cut in half from 128 to 64. The biggest improvements happen in temporal processing, where a lower frame rate and fewer spectral features work together to cut overall complexity by 75%.

We showed that these algorithmic improvements lead to big energy savings in the real world by doing thorough power profiling on a Raspberry Pi 4B. The measurements showed that when modes were switched, power use went down a lot. We measured how much power each mode used in the following ways:

$$P_{full} = 4.3 \text{ W (full resolution)} \quad (23)$$

$$P_{reduced} = 2.5 \text{ W (reduced resolution)} \quad (24)$$

$$\text{Energy Saving} = \frac{P_{full} - P_{reduced}}{P_{full}} = 41.9\% \quad (25)$$

C.8. Integration with Contrastive Learning

A learned correction transform is applied to the output of the reduced-resolution pipeline to make sure that the final feature embeddings are the same in both processing modes. An affine transformation called $T_{correction}$ moves the reduced-resolution embedding ($e_{reduced}$) into the full-resolution feature space. The transform is defined as:

$$e_{corrected} = T_{correction}(e_{reduced}) \quad (26)$$

$$= W_{correction} \cdot e_{reduced} + b_{correction} \quad (27)$$

We learn the weight matrix ($W_{correction}$) and bias vector ($b_{correction}$) by minimizing the L2 distance between the full-resolution embedding and the corrected reduced-resolution embedding over a large dataset. The minimized loss function is

$$L_{correction} = \mathbb{E}_x [\|f_{full}(x) - T_{correction}(f_{reduced}(x))\|_2^2]. \quad (28)$$

The data augmentation techniques are also changed to work with lower-resolution data so that the model training is strong. The parameters of standard methods are changed to take into account changes in the number of dimensions and the data sampling rate. For example, in frequency masking, the width of the mask is made smaller to fit the fewer mel bins. The Time Masking parameters are also changed to fit the bigger hop size. The frequency and temporal decimation factors are usually used to change all of the important parameters for augmentations like SpecAugment. This makes sure that the changes affect both data resolutions in the same way.

D. Details of Edge-Specific Augmentations

The technical details of the lightweight data augmentations $A(x_t)$ used on edge devices to construct positive pairs in StreamSplit are provided in this appendix. In order to maintain the semantic content required for successful contrastive learning, these augmentations are specifically made to be computationally efficient.

D.1. Temporal and Spectral Domain Augmentations

A set of advanced data augmentation methods is used to make the model more generalizable and robust. These changes make realistic changes to the time, frequency, and amplitude domains.

A time-shifting augmentation is used to change the time at first. This method rotates the audio signal around its frame by using a random offset, δ_t , that is picked from the range $[-\tau/4, \tau/4]$, where $\tau=400$ samples. This is what the operation means:

$$A_{time}(x_t)[n] = \begin{cases} x_t[n + \delta_t], & \text{if } 0 \leq n + \delta_t < N \\ 0, & \text{otherwise} \end{cases} \quad (29)$$

To lessen the sudden changes at the edges caused by this shift, a tapering window is put on the edges of the augmented signal. This window has a taper length of $N_{taper} = 40$ samples, which makes sure that the transitions are smooth:

$$w_{taper}[n] = \begin{cases} \sin^2\left(\frac{\pi n}{N_{taper}}\right), & \text{if } n < N_{taper} \\ 1, & \text{if } N_{taper} \leq n < N - N_{taper} \\ \sin^2\left(\frac{\pi(N-n-1)}{N_{taper}}\right), & \text{if } n \geq N - N_{taper} \end{cases} \quad (30)$$

Second, an adaptive frequency masking technique is used to make the system less sensitive to losing spectral information. This addition removes a continuous band of frequencies from the spectrogram:

$$A_{freq}(X_t)[f, \tau] = X_t[f, \tau] \cdot \mathbf{1}_{f \notin [f_0, f_0 + \Delta f]} \quad (31)$$

The spectral energy distribution of the signal itself, not random chance, determines where the mask goes. This makes it more likely that it will cover frequency ranges that are important for perception. Do the following to determine each frequency bin's energy and masking probability:

$$E[f] = \sum_{\tau=0}^{T-1} |X_t[f, \tau]|^2 \quad (32)$$

$$P_{mask}[f] = \frac{E[f]}{\sum_{k=0}^{F-1} E[k]} \cdot \beta_{spectral} \quad (33)$$

The total masking probability is set to $\beta_{spectral} = 0.15$ in this case. Also, the mask's width, Δf , is chosen dynamically to cover 5% to 15% of all frequency bins:

$$\Delta f \sim \text{Uniform}(F \cdot 0.05, F \cdot 0.15) \quad (34)$$

Finally, an amplitude scaling augmentation makes it look like the gain is changing in a realistic way. The signal is modulated by a smooth-changing gain profile, $g[n]$, and a global gain factor, γ :

$$A_{amp}(x_t)[n] = \gamma \cdot x_t[n] \cdot g[n] \quad (35)$$

The gain profile uses a sine wave with a small amplitude ($\alpha_{smooth} = 0.05$) and a random phase ϕ to make small changes over time. The global gain γ comes from a Log-Normal distribution ($\mu = 0, \sigma = 0.1$):

$$g[n] = 1 + \alpha_{smooth} \sin\left(\frac{2\pi n}{N} + \phi\right) \quad (36)$$

To stop audio clipping from too much gain, a dynamic range preservation mechanism is built in. The final gain factor is limited to keep the peak amplitude of the signal within a safe range:

$$\gamma_{final} = \min\left(\gamma, \frac{0.95 \cdot x_{max}}{|\gamma \cdot x_t[n]|_{max}}\right) \quad (37)$$

D.2. Resource-Adaptive Augmentation Strategies

By first setting a computational budget, $\mathcal{C}_{aug}$, the data augmentation strategy changes based on the computing power that is available. This budget is based on the available CPU ($C_{available}$) and memory ($M_{available}$) resources, which are given weights of $\alpha_{cpu} = 0.7$ and $\alpha_{mem} = 0.3$, respectively.

$$\mathcal{C}_{aug} = \alpha_{cpu} \cdot C_{available} + \alpha_{mem} \cdot M_{available} \quad (38)$$

Based on this score, the system picks one of three different augmentation levels: Basic, Standard, or Extended. Each level is best for a different type of resource profile.

$$\text{Aug Level} = \begin{cases} \text{Basic}, & \text{if } \mathcal{C}_{aug} < 0.3 \\ \text{Standard}, & \text{if } 0.3 \leq \mathcal{C}_{aug} < 0.7 \\ \text{Extended}, & \text{if } \mathcal{C}_{aug} \geq 0.7 \end{cases} \quad (39)$$

At each level, the multi-level pipeline adds more and more complicated things. The Basic level is meant for situations with few resources and uses simple time shifting and gain adjustments. It has a low computational cost of $\mathcal{O}(N)$. The Standard level has a cost of $\mathcal{O}(N + F \cdot T)$ and a well-rounded

strategy that includes more advanced techniques like modulated amplitude scaling, adaptive frequency masking, and full time shifting. The Extended level has the highest computational cost of $\mathcal{O}(N + 3F \cdot T)$ and uses the most complicated methods for environments with a lot of resources. These methods include spectral rotation ($X'[f, \tau] = X[(f + \Delta f_{rot}) \bmod F, \tau]$), advanced amplitude modulation, and the use of multiple frequency masks.

Finally, a feedback system that is aware of quality and linked to the model’s training goal keeps making the augmentation policy’s specific parameters, θ_{aug} , better. To reduce the primary contrastive loss, the parameters are changed using a gradient-based method. This helps the model learn to make more useful positive pairs (e_t, e_t^+). The rule for updating is as follows:

$$\theta_{aug}^{(t+1)} = \theta_{aug}^{(t)} - \eta_{aug} \nabla_{\theta_{aug}} L_{contrastive}(e_t, e_t^+) \quad (40)$$

This learning process is guided by an augmented loss function that combines the standard InfoNCE contrastive objective with a regularization term on the augmentation parameters themselves to ensure stability.

$$L_{aug} = -\log \frac{\exp(s(e_t, e_t^+)/\tau)}{\exp(s(e_t, e_t^+)/\tau) + \sum_j \exp(s(e_t, e_j^-)/\tau)} + \lambda_{reg} \|\theta_{aug}\|_2^2 \quad (41)$$

D.3. Implementation Optimizations and Validation

In order to improve computation efficiency, the augmentation pipeline is made to use the least amount of memory possible. This is accomplished by pre-allocating a single temporary buffer at startup to eliminate the additional effort of repeatedly allocating and freeing memory during runtime, and by utilizing transformations in place whenever feasible. The memory-efficient workflow is explained in detail by the following algorithm:

Algorithm 3: Memory-Efficient Augmentation Pipeline

- 1: **Input:** Audio segment x_t , resource level $\mathcal{C}_{aug}$
 - 2: **Output:** Augmented segment x_t^+
 - 3: Allocate temporary buffer $temp[N]$ once
 - 4: Select augmentation level based on $\mathcal{C}_{aug}$
 - 5: **for** each selected augmentation A_i **do**
 - 6: Apply A_i in-place: $x_t \leftarrow A_i(x_t, temp)$
 - 7: Update quality metrics if needed
 - 8: **end for**
 - 9: **return** x_t (now augmented)
-

The effectiveness of this framework was quantitatively tested by examining its impact on the contrastive learning objective using the AudioSet validation set. The results show that the combined augmentation techniques are very successful in producing challenging positive pairs and improving their differentiation from negative pairs. The Combined (Standard) pipeline, for example, maintains a significant contrastive gap of 0.51 between positive (0.76) and negative (0.25) similarities. This strikes the best balance between efficiency and representation quality, and its low computation cost of 2.1ms indicates that it is suitable for use on edge devices.

Table 4: Augmentation effectiveness analysis on AudioSet validation set

Augmentation	Positive Sim.	Negative Sim.	Contrastive Gap	Comp. Cost (ms)
None (Baseline)	0.85	0.31	0.54	0.0
Time Shift Only	0.78	0.29	0.49	0.8
Freq. Mask Only	0.81	0.27	0.54	1.2
Amp. Scale Only	0.82	0.30	0.52	0.5
Combined (Standard)	0.76	0.25	0.51	2.1
Combined (Extended)	0.73	0.22	0.51	4.8

To ensure that the augmentation strategy is effective, it is tested and modified in various audio domains. Speech time shifting is restricted to maintain phoneme boundaries. Frequency masking is altered for music in order to preserve harmonic structures. Environmental sounds employ more aggressive amplitude scaling to simulate microphone changes. By replacing a learned, domain-specific offset ($\Delta\theta_{domain}$) with a base set of parameters (θ_{aug}^{base}), this domain-specific tuning is formally implemented:

$$\theta_{aug}^{domain} = \theta_{aug}^{base} + \Delta\theta_{domain} \quad (42)$$

The contrastive learning performance of StreamSplit is improved by this adaptable and comprehensive framework, which ensures the creation of strong positive pairs while maintaining the computational efficiency required for edge deployment.

E. Derivation and Implementation of Distribution-Aware Sampling (DAS)

To enhance contrastive learning performance, we employ a density-aware negative sampling technique that selects examples from underrepresented regions of the embedding space. This method puts more weight on “hard negatives” that are more helpful. To start, a parametric Gaussian Mixture Model (GMM), $p_\theta(e)$, is used to model the distribution of embeddings that underlies them. This model shows the distribution as a weighted sum of C Gaussian parts:

$$p_\theta(e) = \sum_{c=1}^C \pi_c \mathcal{N}(e; \mu_c, \Sigma_c) \quad (43)$$

Then, based on this learned model, the chance of picking a certain negative example, e_i , is set to be the opposite of its density. This makes it more likely that examples from areas with low density will be chosen. The official definition of sampling probability is:

$$p_{\text{sample}}(e_i) \propto \frac{1}{(p_\theta(e_i) + \epsilon)^\alpha} \quad (44)$$

where ϵ is a small constant that keeps the numbers stable and α is a temperature parameter that controls how sharp the sampling distribution is.

We use an online Expectation-Maximization (EM) algorithm that works with mini-batches of data to keep learning and updating the GMM parameters, $\{\pi_c, \mu_c, \Sigma_c\}_{c=1}^C$. The two steps are done for each batch B :

- *Step E:* For each embedding e_t in the batch, we figure out how much responsibility $\gamma_{t,c}$ each component c has. This

value shows the posterior probability that component c made e_t .

$$\gamma_{t,c} = \frac{\pi_c \mathcal{N}(e_t; \mu_c, \Sigma_c)}{\sum_{j=1}^C \pi_j \mathcal{N}(e_t; \mu_j, \Sigma_j)} \quad (45)$$

- *M-step*: The GMM parameters are updated using a smoothed rule with a learning rate β . By interpolating between the new estimates from the current batch and the old parameters, this makes sure that online adaptation stays stable. The update equations are:

$$\mu_c^{(t+1)} = (1 - \beta)\mu_c^{(t)} + \beta \frac{\sum_{i \in B} \gamma_{i,c} e_i}{\sum_{i \in B} \gamma_{i,c}} \quad (46)$$

$$\Sigma_c^{(t+1)} = (1 - \beta)\Sigma_c^{(t)} + \beta \frac{\sum_{i \in B} \gamma_{i,c} (e_i - \mu_c^{(t+1)})(e_i - \mu_c^{(t+1)})^T}{\sum_{i \in B} \gamma_{i,c}} \quad (47)$$

$$\pi_c^{(t+1)} = (1 - \beta)\pi_c^{(t)} + \beta \frac{1}{|B|} \sum_{i \in B} \gamma_{i,c} \quad (48)$$

F. Sensitivity Analysis for Hybrid Loss Hyperparameter λ

The hyperparameter λ in the server-side hybrid loss function, $\mathcal{L}_{\text{server}} = \mathcal{L}_{\text{SW}} + \lambda \mathcal{L}_{\text{Lap}}$, balances the global distribution alignment from the Sliced-Wasserstein distance and the local structure preservation from the Laplacian regularization. We did a parameter sweep for λ on the AudioSet (Subsampled) validation set to find the best value and see how sensitive the model is. We tried out different values of λ , from 10^{-4} to 10^1 . The model’s performance is stable across a wide range of λ values, from 10^{-2} to 1, as shown in Figure 1. This is shown by the accuracy of downstream classification. The best results came from using $\lambda = 0.1$ as the default value for all tests. When λ is less than 10^{-3} , the Laplacian term doesn’t have much of an effect on performance. This is the same as just using the Sliced-Wasserstein loss. When λ is very large (greater than 1), the model puts too much emphasis on local structure. This can make it less accurate and harder for it to form clusters that make sense globally.

G. Proof Sketch for Theorem 1

The convergence analysis for the non-convex case begins with the standard descent lemma for an $\mathcal{L}$ -smooth function $\mathcal{L}$, which establishes a constraint on the extent of progress achievable after a single update step:

$$\mathcal{L}(f_{t+1}) \leq \mathcal{L}(f_t) - \eta \langle \nabla \mathcal{L}(f_t), \tilde{\nabla} \mathcal{L}(f_t) \rangle + \frac{L\eta^2}{2} \|\tilde{\nabla} \mathcal{L}(f_t)\|^2 \quad (49)$$

$\tilde{\nabla} \mathcal{L}(f_t)$ Then we find the expected value of the stochasticity of the gradient estimate $\tilde{\nabla} \mathcal{L}(f_t)$ and the extra error that comes from the split approximation. If the gradient is not biased and has a variance of σ^2 , and the error in the split approximation is not more than ϵ^2 , then the expected progress is:

$$\begin{aligned} \mathbb{E}[\mathcal{L}(f_{t+1})] &\leq \mathbb{E}[\mathcal{L}(f_t)] - \eta \|\nabla \mathcal{L}(f_t)\|^2 \\ &\quad + \frac{L\eta^2}{2} (\|\nabla \mathcal{L}(f_t)\|^2 + \sigma^2) + L\epsilon^2 \end{aligned} \quad (50)$$

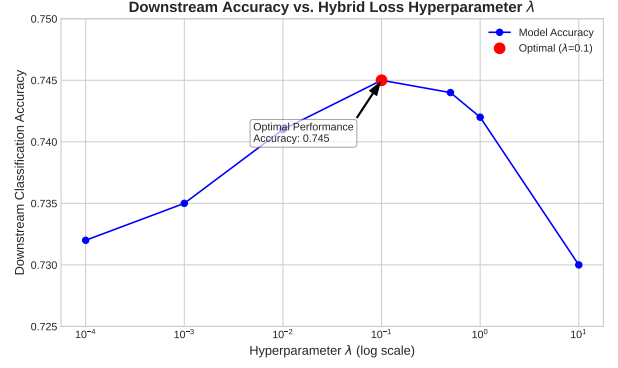


Figure 1: The downstream classification accuracy on the AudioSet (Subsampled) validation set changes when the hybrid loss hyperparameter λ changes. The best performance happens around $\lambda = 0.1$.

The loss function terms on the right side make a telescoping sum when you rearrange the terms to isolate the squared gradient norm and add the inequality from $t=0$ to $T-1$. This gives us the following limit on the sum of the expected squared gradients:

$$\begin{aligned} \eta \sum_{t=0}^{T-1} \left(1 - \frac{L\eta}{2}\right) \mathbb{E}[\|\nabla \mathcal{L}(f_t)\|^2] &\leq \mathcal{L}(f_0) - \mathbb{E}[\mathcal{L}(f_T)] \\ &\quad + \frac{TL\eta^2\sigma^2}{2} + TL\epsilon^2 \end{aligned} \quad (51)$$

Finally, we can limit the average expected squared gradient norm by picking a learning rate $\eta < 2/L$ to make sure the left side is positive. Finding the minimum over all iterations shows that the algorithm will always find a point where the expected gradient is small. This means that it will converge to a point that doesn’t move.

H. HeuristicSplit Baseline Implementation Details

The *HeuristicSplit* baseline is a dynamic replacement for our RL agent that does not learn. It uses a few simple, hand-made rules to figure out where to break up the MobileNetV3-Small architecture. The rules depend on two main things: how much CPU is being used right now and how much bandwidth the network is expected to have. This is how you set up the heuristic function:

$$\text{SplitLayer} = \text{clip}(\lfloor L \cdot (w_{cpu} \cdot (1 - U_{cpu}) + w_{bw} \cdot B_{norm}) \rfloor, 1, L - 1) \quad (52)$$

where L is the total number of layers, U_{cpu} is the CPU utilization (0-1), and B_{norm} is the normalized bandwidth (0-1). The weights $w_{cpu} = 0.6$ and $w_{bw} = 0.4$ were set so that offloading would be the most important thing to do when the device is busy. This rule-based system is more flexible than FSL, but it doesn’t have the foresight or ability to find the best long-term rewards that the RL agent in StreamSplit does.

I. On-Device Dataset Details

We created and gathered a unique audio dataset on the device in order to test the StreamSplit system in an actual edge deployment environment. The data collection process involved 48 hours of continuous audio recording using four Raspberry Pi 4 devices that resembled the target hardware platform. These devices were positioned in a variety of locations to capture a wide range of real-world sound conditions, including a city apartment balcony, a busy kitchen, a quiet home office, and a living room with media playback. All of the audio was captured using standard USB microphones with a 16-bit resolution and a sampling rate of 16 kHz. The continuous recordings were divided into 10-second segments and labeled by three separate annotators. A majority vote determined the final label for every clip. Conversation, Background Music, Silence, Footsteps, Keyboard Typing, Appliance Noise, and Outdoor Traffic are the seven classes that make up the dataset. Ultimately, the dataset was separated into three sets: testing (10%), validation (10%), and training (80%). To ensure that clips from the same recording session were not divided between sets, extra care was taken. This ensured a robust evaluation and prevented data leaks.

J. Evaluation Metrics

A set of metrics that looked at both the practical performance on edge devices and the quality of the learned representations was used to give a full picture of the system. To see how useful the features were, we used Downstream Classification Accuracy, which shows the top-1 accuracy of linear probes trained on the frozen representations. We used a Nearest Neighbor Retrieval task to look at the structure of the embedding space, and we report Precision@10 based on cosine similarity.

Three important metrics for edge computing were used to measure the performance of the system as a whole. The communication cost was based on the total number of megabytes (MB) of data sent from the edge device to the server every hour. Latency was measured from the time an audio clip was recorded to the time it was processed. An external power meter was used to measure the Raspberry Pi 4’s energy use in Watts (W) over one-hour runs. Finally, the Silhouette Score was used to measure cluster density and separation. A score close to 1 means that the clusters are well-defined. This was done to quantitatively assess the quality of embedding space visualizations.

K. Baseline and Hyperparameter Tuning Details

We made sure that all of the models, including StreamSplit and the baselines, could be fairly compared by using the same hyperparameter tuning method. We used a three-fold cross-validation grid search to maximize the validation split of our subsampled AudioSet dataset. The ideal set of hyperparameters for each model was identified by comparing the configuration that produced the highest average downstream classification accuracy over the three folds. The batch size

(from $\{32, 64, 128, 256\}$), the contrastive learning temperature (from $\{0.07, 0.1, 0.2, 0.5\}$), and the learning rate (from $\{1e-5, 5e-5, 1e-4, 5e-4\}$) were all included in the search space for each model.

For situations with limited resources, we created the *LEO* (Lightweight Edge-Only) baseline. An edge device can be used to fully train this model’s efficient architecture. The LEO architecture is a modified MobileNetV2-Tiny that is created by applying a width multiplier of 0.35 to a standard MobileNetV2. Its size was further reduced by halving the number of channels in its inverted residual blocks compared to the MobileNetV3-Small architecture used in our main model. These changes make the model leaner and are intended to use the least amount of CPU and memory, with about 0.5 million parameters.

L. Network Simulation Methodology

We used the Linux Traffic Control (‘tc’) tool to simulate the connection between the edge device and the server. This was done to make sure that the system performance test was realistic and could be done again and again under different network conditions. This tool was installed on the server’s network interface to dynamically shape traffic using a script that was run from a trace file. To make the simulation more realistic, this trace file was made using an open-source dataset of measurements from urban 4G/LTE mobile networks. The script changed the network settings in a planned way. The latency went from 50ms to 200ms, and the bandwidth went from 0.5Mbps to 8Mbps. A 15ms packet jitter was also added to make it more realistic. The system’s communication resilience was tested in a dynamic setting by changing these network conditions every 30 seconds to make it seem like a user was moving through areas with different levels of connectivity.

M. Laplacian Regularization Details

For every mini-batch of embeddings, $\{e_i\}$, received by the server, a Laplacian regularization term, $\mathcal{L}_{\text{Lap}}$, is generated. This preserves the manifold structure of the embedding space and encourages local smoothness. The first step is to create a k-nearest neighbor (k-NN) graph using the embeddings. For every test, k was set to 10. The affinity matrix W for the graph is then determined using a Gaussian kernel. This gives all other neighbors a weight of zero and connected neighbors e_i and e_j a weight of $W_{ij} = \exp(-\|e_i - e_j\|^2 / 2\sigma_W^2)$. We set the kernel’s bandwidth parameter, σ_W , for each batch as the median distance between all pairs in the k-NN graph to guarantee that this computation continues even if the embeddings in different batches have varying sizes. Lastly, by subtracting D and W from one another, we obtain the unnormalized graph Laplacian, L . With elements defined as $D_{ii} = \sum_j W_{ij}$, the affinity matrix is W and the diagonal degree matrix is D .

N. RL Agent and Architecture Transformation Details

The system uses a reinforcement learning (RL) agent to figure out when to split the work between the edge and the

server in real time. It does this using the Proximal Policy Optimization (PPO) algorithm. The agent’s choices are based on a full state space, s_t , which has normalized values for on-device metrics (CPU usage, memory usage, battery level), current network conditions (bandwidth, latency), and an embedding uncertainty measure that comes from the server. The agent’s discrete action space, a_t , shows all the possible layers in the MobileNetV3 backbone where a split could happen. It also includes an action for doing all the calculations on the edge. $R = w_{acc} \cdot \Delta_{acc} - w_{lat} \cdot \text{latency} - w_{eng} \cdot \text{energy}$ is a weighted sum that teaches the agent its policy by using weights of $w_{acc} = 1.0$, $w_{lat} = 0.3$, and $w_{eng} = 0.2$. It compares the increase in downstream accuracy (Δ_{acc}) to the costs of latency and energy that have been normalized.

Before being sent to the server, the intermediate data goes through dynamic architecture changes to cut down on communication overhead after the RL agent picks a split point. The main method, called bottleneck insertion, involves adding a small, fully connected layer at the point where the two parts meet. This layer’s training cuts the intermediate activation’s dimensions by 75%, which lets the server-side model rebuild the original activation with very little information loss. A more aggressive Layer Quantization technique is used when network bandwidth is very limited. This method lowers the data payload even more by using standard symmetric quantization to change the activations from 32-bit floating-point numbers to 8-bit integers.

O. Training and Implementation

Hyperparameters

A set of hyperparameters makes the on-device training process fast and stable, even on hardware with limited resources. For the main contrastive learning algorithm, the consistency loss weight was set to $\lambda_{\text{consistency}} = 0.5$, and for the key encoder, the momentum coefficient was set to $m = 0.999$. Gradient accumulation was done over $N = 4$ mini-batches before each weight update to make it look like there were more batches. We used an adaptive learning rate that changed based on CPU usage, starting at a base rate of $\eta = 1e - 4$. When the CPU usage goes over 80%, the learning rate can drop by as much as 50%. The Distribution-Aware Sampling (DAS) module was set up with a 5-component GMM, a stability constant of $\epsilon = 1e - 6$, a sampling temperature of $\alpha = 0.5$, and an EM update learning rate of $\beta = 0.1$.

The server-side implementation has a number of adaptive features that help manage the distributed learning pipeline. Every time a new batch of embeddings comes in from edge devices, a mini-batch k-means algorithm updates a set of $k = 50$ prototype centers. The weights of the multi-component uncertainty metric are first set to the same value. Then, they are changed using a simple reinforcement mechanism that rewards the source that is most likely to make high-loss embeddings. The adaptive uncertainty threshold uses the 70th percentile of uncertainty values from a warm-up phase as its baseline, or $\mathcal{U}_{\text{base}}$. Then, it is changed based on the state of the network and battery using weighting parameters $\beta_1 = 0.5$ and $\beta_2 = 0.5$. For temporal aggregation

tasks, the Gaussian kernel bandwidth, σ , is set adaptively to the median distance of embeddings within each received window to make sure that it can handle changes in embedding density.

References

Borgerding, M. 2018. GitHub - mborgerding/kissfft: a Fast Fourier Transform (FFT) library that tries to Keep it Simple, Stupid — github.com. <https://github.com/mborgerding/kissfft>.